Computer Vision Project Final Report

Object Detection and Segmentation on Amazon ARMBench

Group Members:

1) Nithish Krishna Shreenevasan (nshreen1@jh.edu)
2) Rahul Arutla (rarutla1@jh.edu)
3) Swastid Kasture (skastur3@jh.edu)

Introduction:

This project's purpose is to use a set of recent deep learning models to detect objects and segment instances on the Amazon ARMBench dataset. Unlike standard approaches, which treat detection and segmentation as separate jobs, this study investigates integrated systems capable of performing both at the same time. Object detection models such as YOLO, R-CNN, and their variants are primarily concerned with recognizing and localizing objects inside an image. Meanwhile, segmentation models such as U-Net, DeepLabV3+, and Kite-Net excel at pixel-level classification but frequently necessitate distinct processes for object localization.

To close this gap, we use Mask R-CNN, a model that extends Faster R-CNN by including a specialized segmentation branch with the standard classification and bounding box regression heads. This enables end-to-end object detection and instance-level segmentation. The Amazon ARMBench dataset presents a demanding test with congested landscapes and a variety of objects, making it ideal for evaluating such models.

Several recent studies have advanced object segmentation, particularly in settings involving unseen objects and robotic applications. ZISVFM proposes a zero-shot segmentation strategy that uses the vision foundation models DINOv2 and SAM (Segment Anything Model) to segment objects without prior training. UOIS-Net uses synthetic RGB-D data to partition previously undetected object instances, relying on depth-based mask suggestions modified by RGB features. STOW employs a transformer-based approach for inter-frame communication to segment and track unseen objects over temporally dispersed discrete frames.

In our project, we evaluate the performance of Mask R-CNN, DeepLabV3+, RetinaNet, and YOLOv8 on the ARMBench object segmentation track. By assessing various models under identical conditions, we want to establish which architectures are most effective for robotic object understanding in real-world, crowded contexts.

Methods:

Data:

This study focuses on the use of instance segmentation, a computer vision approach for detecting and distinguishing discrete objects inside an image. This is especially useful in warehouse automation, where precisely recognizing and locating things in containers is critical for subsequent robotic activities like object recognition and grab planning. Instance segmentation provides precise manipulation by providing pixel-level annotations to aid robotic interaction.

To assess model performance and transferability in different visual situations, we employ the Amazon ARMBench dataset's Object Segmentation track. The dataset is divided into three subsets. The first subset, Mix-Object-Tote (14 GB), has 44,253 high-resolution (2448×2048 pixels) close-up photos of mixed objects housed in yellow or blue totes. It has 467,225 annotations, with an average of 10.5 object instances per image. The second subset, Zoomed-Out-Tote-Transfer-Set (1.5 GB), contains 5,837 photos (2046×2046 pixels) shot from a larger distance under varying lighting conditions. The photographs contain 43,401 annotations and an average of 7.5 instances per tote.

The third subset, Same-Object-Transfer-Set (3 GB), includes 3,323 photos (2048×1500 pixels) with many instances of the same object positioned close together. There are 12,664 annotations and an average of 3.8 objects per scene. The dataset annotations are given in LabelMe and MS-COCO formats, with relevant train, validation, and test splits included.

Methods and Algorithms:

We compare three model architectures, for instance segmentation: Mask R-CNN, DeepLabV3+ with RetinaNet pipeline, and YOLOv8.

1) Mask RCNN: Mask R-CNN builds on Faster R-CNN by adding a parallel branch for predicting object masks in addition to bounding boxes. It enables exact instance segmentation by recognizing object positions while also generating high-quality pixel-wise masks. Its two-stage structure performs well on benchmark datasets and is commonly used for precise segmentation tasks.

2) DeepLabV3+ and Retina Net: RetinaNet is a one-stage object detector that use Focal Loss to address class disparities between foreground and background items. It applies detection densely throughout the image, achieving accuracy comparable to two-stage models while being faster. We integrate it with DeepLabV3+, a semantic segmentation model that uses atrous spatial pyramid pooling and a decoder module to collect multi-

scale context and revise object boundaries, resulting in improved segmentation results.

3) YOLOv8: YOLOv8 is a real-time object detection and segmentation model that combines detection and mask prediction in a single stage architecture. It is lightweight and deployment-optimized, making it ideal for robotics applications requiring low latency.

Software Used:

All models and experiments were built using PyTorch. We employ the official PyTorch implementations of Mask R-CNN, DeepLabV3+, and RetinaNet, as well as publicly available third-party implementations of YOLOv8. The dataset was processed with custom-made PyTorch dataset classes and dataloaders. Because the dataset is formatted in MS-COCO style, we use the pycocotools module to load annotations and compute evaluation metrics like mean Average Precision (mAP).

All training and evaluation activities were completed on Google Colab Pro, which utilized T4 GPUs to expedite training. We utilize the matplotlib and OpenCV tools to visualize ground truth and expected masks for qualitative analysis.

Results:

To compare the performance of the various models on the same-object transfer task, we used several cutting-edge architectures for object identification and instance segmentation: Mask R-CNN, DeepLabV3, RetinaNet, and YOLOv8. We use quantitative measurements (such as mAP, IoU, precision, recall, and F1 score) and qualitative visualizations to compare the models' accuracy and segmentation quality. The findings show how the models trade off speed, accuracy, and localization capabilities.

1) **Mask RCNN Results:**

ResNet50 Backbone

The following tables summarize the bounding box and segmentation mask results obtained for Mask RCNN with a Resnet 50 backbone.

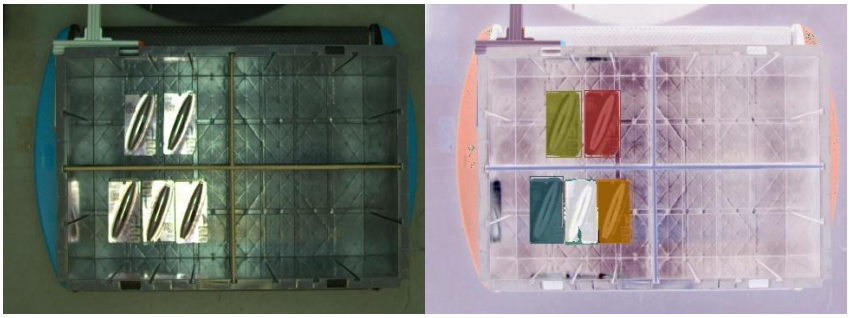
Bounding Box

Metric	IoU Threshold	maxDets	Value
Average Precision	0.5:0.95	100	**0.812**
Average Precision	0.5	100	**0.965**
Average Precision	0.75	100	**0.914**
Average Recall	0.5:0.95	1	**0.242**
Average Recall	0.5:0.95	10	**0.810**
Average Recall	0.5:0.95	100	**0.856**

Segmentation Masks

Metric	IoU Threshold	maxDets	Value
Average Precision	0.5:0.95	100	**0.854**
Average Precision	0.5	100	**0.955**
Average Precision	0.75	100	**0.926**
Average Recall	0.5:0.95	1	**0.250**
Average Recall	0.5:0.95	10	**0.844**
Average Recall	0.5:0.95	100	**0.890**

Below is an example of the model during inference.



ResNet18 Backbone

The following tables summarize the bounding box and segmentation mask results obtained for Mask RCNN with a Resnet 18 backbone.

Bounding Box

Metric	IoU Threshold	maxDets	Value
Average Precision	0.5:0.95	100	**0.594**
Average Precision	0.5	100	**0.877**
Average Precision	0.75	100	**0.690**
Average Recall	0.5:0.95	1	**0.182**
Average Recall	0.5:0.95	10	**0.651**
Average Recall	0.5:0.95	100	**0.709**

Segmentation Masks

Metric	IoU Threshold	maxDets	Value
Average Precision	0.5:0.95	100	**0.654**
Average Precision	0.5	100	**0.865**
Average Precision	0.75	100	**0.764**
Average Recall	0.5:0.95	1	**0.198**
Average Recall	0.5:0.95	10	**0.707**
Average Recall	0.5:0.95	100	**0.768**

Below is an example of the model outputs during inference.

ResNet101 Backbone

The following tables summarize the bounding box and segmentation mask results obtained for Mask RCNN with a Resnet 101 backbone.

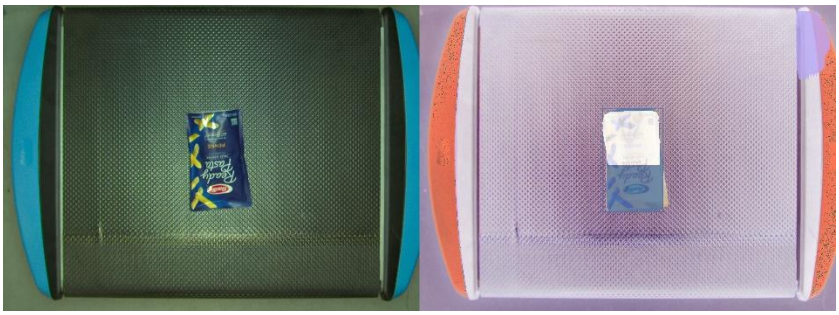
Bounding Box

Metric	IoU Threshold	maxDets	Value
Average Precision	0.5:0.95	100	**0.643**
Average Precision	0.5	100	**0.874**
Average Precision	0.75	100	**0.747**
Average Recall	0.5:0.95	1	**0.203**
Average Recall	0.5:0.95	10	**0.708**
Average Recall	0.5:0.95	100	**0.750**

Segmentation Masks

Metric	IoU Threshold	maxDets	Value
Average Precision	0.5:0.95	100	**0.705**
Average Precision	0.5	100	**0.869**
Average Precision	0.75	100	**0.811**
Average Recall	0.5:0.95	1	**0.224**
Average Recall	0.5:0.95	10	**0.767**
Average Recall	0.5:0.95	100	**0.812**

Below is a picture of the model outputs during inference.



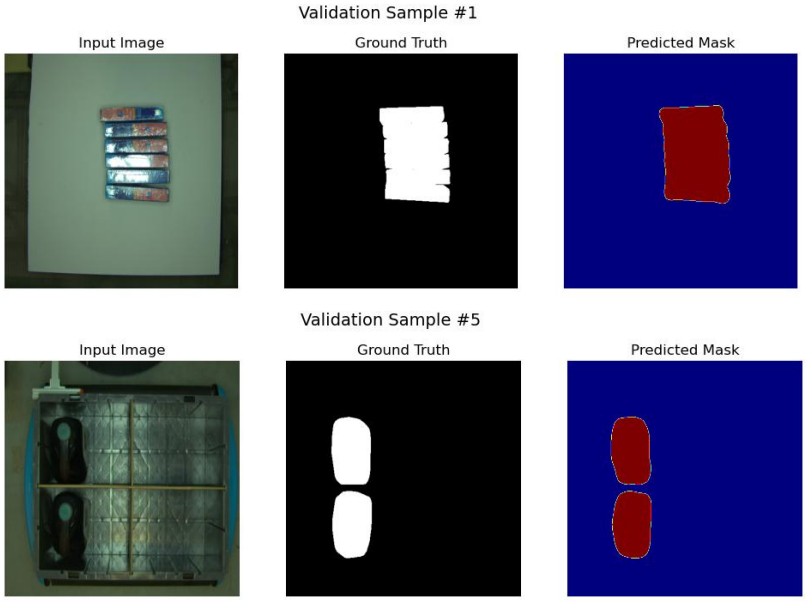
2) DeepLabv3 and Retina Net Results:

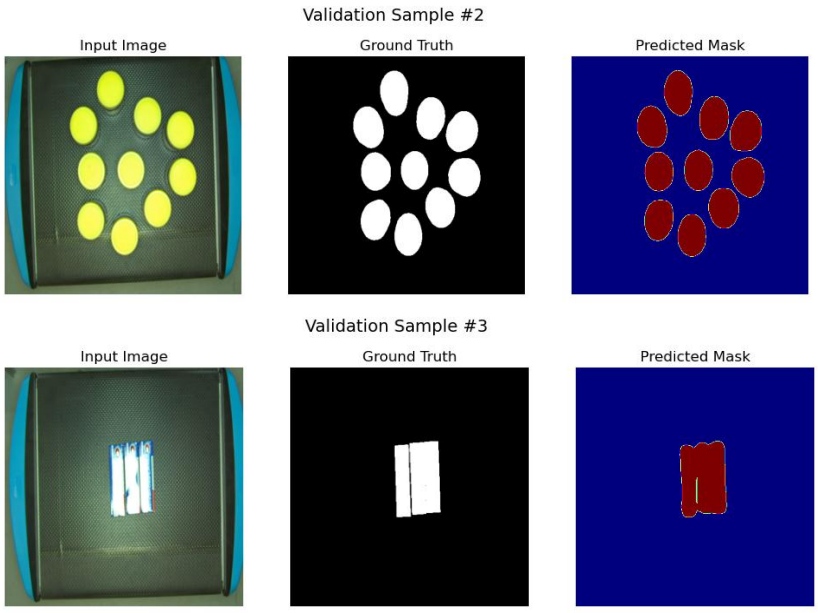
DeepLabv3

For DeepLabv3, the Mean IoU was calculated for evaluation.

Dataset Split	Mean IoU
Validation	**0.9609**
Test	**0.9604**

Below are some pictures of the model outputs obtained during inference.





RetinaNet

The table below summarizes the results obtained by training retina net on the dataset.

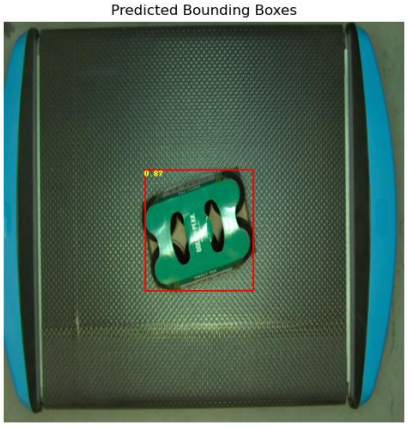
Bounding Box

Metric	IoU Threshold	maxDets	Value
Average Precision	0.5:0.95	100	**0.618**
Average Precision	0.5	100	**0.931**
Average Precision	0.75	100	**0.682**
Average Recall	0.5:0.95	1	**0.216**
Average Recall	0.5:0.95	10	**0.685**
Average Recall	0.5:0.95	100	**0.721**

Below are some pictures obtained using the model at inference.

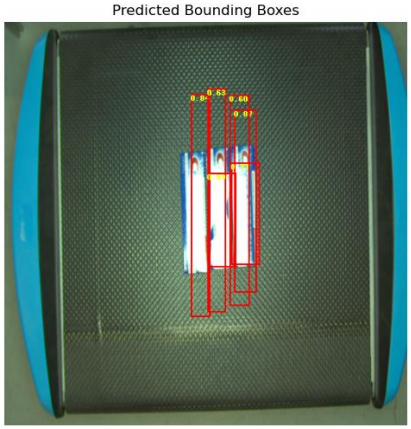
Original Image

Predicted Bounding Boxes



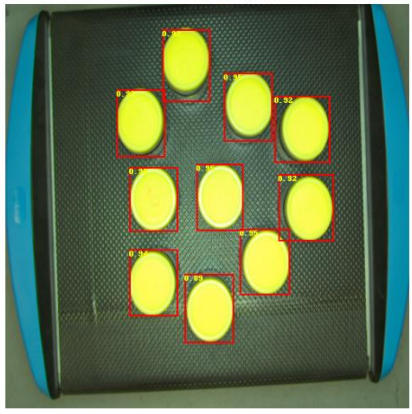
Original Image

Predicted Bounding Boxes



Original Image

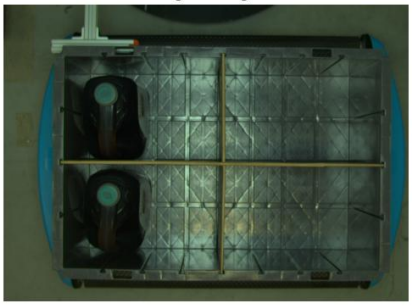
Predicted Bounding Boxes



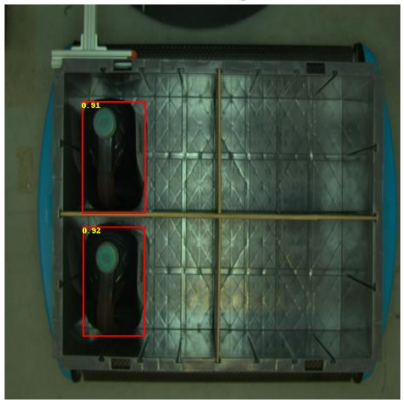
Original Image

Predicted Bounding Boxes

Original Image



Predicted Bounding Boxes

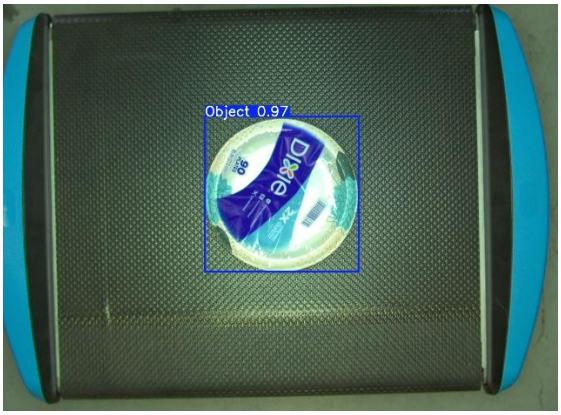


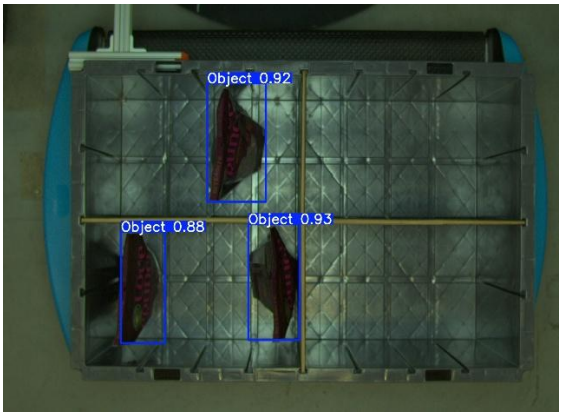
3) Yolov8

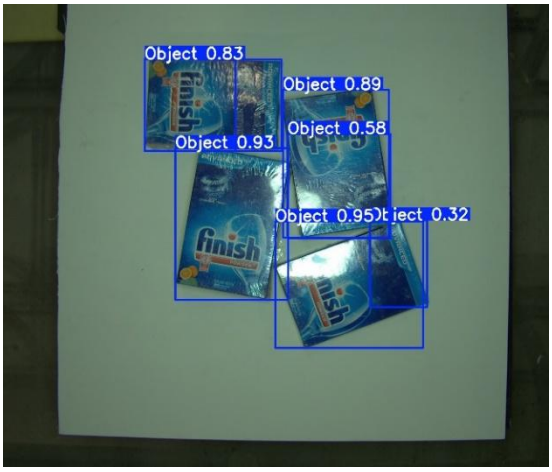
The following tables summarize the results obtained on the validation and test splits of the datasets.

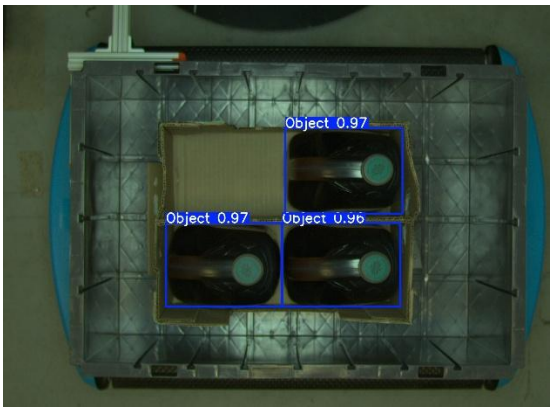
Dataset Split	Precision	Recall	Map50	Map50:95
Validation	**0.973**	**0.967**	**0.987**	**0.907**
Test	**0.865**	**0.942**	**0.885**	**0.797**

Here are some pictures illustrating the qualitative performance of the model during inference.






Object 0.83
Object 0.89
Object 0.58
Object 0.93
Object 0.95
Object 0.32
finish


Object 0.97
Object 0.97
Object 0.96


Object 0.96
Fiskies

Discussions:

1) **Summary of results:**

For instance segmentation and object detection tasks, Mask R-CNN with a ResNet-50 backbone performed the most consistently and dependably out of all the models that were examined. While different backbones were examined, ResNet-50 achieved the optimal balance, resulting in high segmentation quality. In contrast, the ResNet-101 backbone showed evidence of underfitting and was unable to generate appropriate segmentation masks. However, bounding box predictions were consistent across all Mask R-CNN variations, demonstrating that the underlying Faster R-CNN architecture is well-suited to detection on this dataset.

DeepLabv3 demonstrated exceptional segmentation performance, with a mean IoU of 96% across both validation and test splits. Although it conducts semantic segmentation rather than instance segmentation, the quality of the masks suggests that it is quite effective for jobs that do not require object-level separation. RetinaNet performed reasonably well, with a mean average precision of 61.8%. It occasionally failed to recognize several items in congested situations, indicating difficulties with occlusion or thick object layouts.

YOLOv8 fared well in object detection, with a precision of 97.3% on the validation set and 86.5% on the test set. YOLOv8 outperformed RetinaNet and matched Faster R-CNN in constructing bounding boxes for objects in congested surroundings, giving it a viable option for real-time and high-density detection applications.

2) **How do our results compare to other people's work?**

The majority of the models we tested, Mask R-CNN, DeepLabv3, RetinaNet, and YOLOv8, were trained in deterministic environments where the object classifications were predetermined. While these models work well in controlled settings, real-world applications frequently require the ability to recognize and segment previously undetected things. Recent techniques, such as SAM and STOW, overcome this issue by allowing zero- or few-shot instance segmentation, making them better suitable for dynamic contexts like robotic warehouses. Similarly, ZISVFM uses vision foundation models to provide zero-shot object segmentation in indoor robotics. Conversations with the course instructor showed that companies like Amazon may adopt transformer-based models, such as SWIN, for similar jobs. Thus, while our trained models are fast and accurate in bounded domains with little

object variability, future systems may benefit from incorporating adaptive models for larger generalization in open-set settings.

3) Future Work

To expand on this research, the best-performing models revealed in this study will be tested on the mixed-object and zoomed-out-transfer datasets to determine their robustness to more complicated and varied input settings. These further tests will verify whether the models can effectively generalize to varied item configurations and scales. In the long run, once optimal models for detection and segmentation are established, they might be integrated into a real-time robotic system. When combined with an item classification module, the robot will be able to automatically recognize, segment, and sort objects from a tray into their proper packages, paving the way for a comprehensive end-to-end automation process.

Conclusion

Based on the results, it is clear that several models perform well in object recognition and segmentation tasks on the same object transfer dataset. For object detection, Faster R-CNN and YOLOv8 are also feasible choices. While Faster R-CNN exhibits consistent bounding box performance across backbones, YOLOv8 provides competitive precision, particularly in congested settings, and benefits from faster inference speeds, making it an appealing option for real-time applications. When it comes to segmentation, Mask R-CNN is the best model for jobs that require both bounding boxes and instance-specific segmentation masks. Among its backbones, ResNet-50 provides the finest balance of accuracy and generalization. However, if segmentation speed is more important than instance-level detail, DeepLabv3 emerges as a strong contender, producing extremely accurate semantic masks with an average IoU of 96% across validation and test splits. In summary, the model chosen is determined by the deployment scenario's specific requirements. Mask R-CNN with ResNet-50 is the ideal algorithm for high-accuracy instance segmentation. For lightweight, quick inference pipelines, integrating DeepLabv3 for segmentation with YOLOv8 for detection may be preferable. This adaptability enables practitioners to tailor the vision system to diverse real-world robotics and automation tasks depending on performance and computational trade-offs.

Contribution

Nithish (Me) - Nithish played a central role in the execution and coordination of the project. He was responsible for developing and managing the Mask R-CNN pipeline, which involved training and testing the model and building custom dataloaders compatible with the COCO API. He also compiled and authored the final report, ensuring all aspects of the project were well-documented. In addition to his technical contributions, Nithish led the team's collaborative efforts by organizing regular meetings, coordinating model exploration across the team, and guiding the next steps throughout the project timeline.

Rahul - Rahul focused extensively on the DeepLabv3 and RetinaNet components of the project. His work involved modifying the dataset format to be compatible with the COCO API and implementing training and evaluation pipelines for both models. Rahul contributed crucial results including performance metrics and qualitative outputs such as bounding box predictions and segmentation masks. He also actively participated in team meetings and provided timely updates on his progress, facilitating smooth integration of his work into the overall project.

Swastid - Swastid led the YOLOv8 implementation effort, which included reformatting the dataset to comply with YOLO's API's and training the detection model. He ensured reproducibility by sharing detailed codebooks and a ZIP file containing 100 images with predicted bounding boxes. Swastid also participated in team meetings and communicated progress effectively, allowing the group to include his results in the final comparative analysis and visual evaluations.